
CSE425 Neural Network Project Report

Thamim Nur Zahin

Department of Mathematics and Physical Science
BRAC University
Dhaka, Bangladesh
thamim.nur.zahin@g.bracu.ac.bd

Araf Hasan

Department of Computer Science
BRAC University
Dhaka, Bangladesh
araf.hasan@g.bracu.ac.bd

Abstract

This project explores how to teach an AI to write multi-genre symphonic music. Writing music is difficult for computers because it requires long-term memory, pattern recognition, smoothing sound etc. To solve this, we built multiple deep learning models, starting from random sound generators to highly complex "Transformer" model. Our best model, the Task 3 Transformer, successfully learned the rules of music, scoring a great Perplexity of 5.31 and earning high marks from human listeners (from a very small sample set). However, when we tried to push the AI even further in Task 4 by using human feedback (RLHF), the model broke down. It found a loophole in the math and started looping the same sounds just to get a high score, proving that advanced AI training methods can actually destroy a good model if you do not give it enough human data.

1 Introduction

Music is a delicate balance of strict mathematical patterns and creative randomization. For a long time, getting computers to generate music that actually sounds like human art has been a massive challenge. If a computer only looks at the previous note to guess the next one, the song quickly loses its rhythm and turns into a mess.

The goal of this project was to see if we could build a neural network model that understands the "big picture" of a song. To do this, we tested four models: an Autoencoder, a VAE, a Transformer, and an RLHF-Tuned Model.

Throughout this report, we will see that a naive Autoencoder struggles to keep a beat, resulting in a loose tempo and awkward moments of silence. Upgrading to a VAE works much better, producing stable music with recognizable patterns. Our Transformer achieves the peak result, fixing the awkward silences and generating strong, continuous rhythm. Finally, we will see how the RLHF model breaks down completely, resulting in a wall of random noise with too many instruments playing at once. Interestingly, the RLHF failure is the exact opposite of the Autoencoder's failure, proving that in algorithmic music, too much noise is just as bad as too much silence.

2 Methodology (Detailed)

To build a system that understands the "big picture" of music, we combined three major datasets: Maestro (classical piano), Lakh (multi-genre), and Groove (human drumming). Our process started with a deep look at the data before growing our AI models.

Data Preparation & EDA: We performed an Exploratory Data Analysis (EDA) on the Maestro dataset, which contains over 200 hours of professional piano performances. We found that the tracks were extremely long and complex, so we compressed them into manageable segments. To help the model learn, we used the REMI tokenizer, which turns music into a series of words describing

the "Pitch," "Velocity" (loudness), and "TimeShift" (rhythm) of every note. We also optimized our memory usage by sampling 2,000 files and folding all notes into a 5-octave range (C2 to C7) to keep the AI focused on the most important musical zones. It is important to note that before each task, the model has their own EDA too for better performance.

The Baselines (Random & Markov): We started with a Random Generator that just smashed keys with no logic. Then we built a Markov Chain, which looks at what note is playing and guesses the most common note that comes next. These were our "worst-case scenario" tests.

Task 1: We used an "Encoder" to read the music and turn it into a math vector, and a "Decoder" to turn that vector back into notes. Because we used Long Short-Term Memory (LSTM) layers, the model could "remember" the rhythm for short periods, but it often struggled with long-term song structure.

Task 2: Unlike a regular Autoencoder, the VAE organizes its memory into a smooth "Latent Space". This allowed us to add Genre Conditioning. By feeding the model a "Genre ID" during training, we taught it to distinguish between different styles, allowing us to generate music that followed a specific genre's rules. The proper distribution made the music more consistent.

Task 3: The Transformer was our most advanced model. Instead of reading note-by-note, the Transformer uses Self-Attention and a Causal Mask to look at the entire sequence at once and predict the next note. To make sure the Transformer didn't just repeat the same note forever or create long, awkward silences, we built a "Musical Rulebook" directly into the generator, which is directly inspired from music theory. These Heuristics are:

Task 3: The Transformer was our most advanced model. Instead of reading note-by-note, the Transformer uses Self-Attention and a Causal Mask to look at the entire sequence at once and predict the next note. To make sure the Transformer didn't just repeat the same note forever or create long, awkward silences, we built a "Musical Rulebook" directly into the generator, which is directly inspired from music theory. These Heuristics are:

- **Tonal Biasing (Diatonic Bias):** To ensure the generated music stayed in a pleasant key, we applied a mathematical boost to diatonic notes (specifically within the C-Major/A-Minor scales) and heavily penalized out-of-key dissonant notes.
- **The Silence Nuke (Long Pause Tax):** We noticed the model sometimes liked to stay silent for too long. We added a penalty that made "silence tokens" mathematically expensive, forcing the AI to keep the rhythm moving.
- **Reverse Momentum (Anti-Repetition):** If the model played the same note too many times, we applied a penalty to that specific note. This forced the AI to explore new melodies instead of getting stuck on one sound.
- **Octave Fatigue:** To keep the music from sounding piercing or annoying, we tracked how long the model stayed in a specific high or low range. If it stayed in one "octave" for more than 32 notes, we penalized that range to encourage it to move to a different part of the piano.
- **The 3-Octave Cage:** We created a boundary so that the AI couldn't jump across the entire keyboard in one second. This kept the music within a range that a human hand could actually play.

Task 4: Finally, we tried Reinforcement Learning from Human Feedback (RLHF). We used a small survey where humans rated tracks from 1 to 5. We used the REINFORCE algorithm to give the model a "reward" for making music that sounded like the high-rated tracks. Note that as this was an extension of the previous task, all the heuristics of task 3 were used here too.

3 Result Analysis

3.1 Comparison

To understand how our AI grew from a simple random guesser into a sophisticated music generator, we need to look at the final evaluation results. Table 1 provides a complete summary of every model's

Table 1: Performance Comparison Across All Generative Models

Model	Loss	Perplexity	Rhythm Diversity	Repetition Ratio	Human Score	Genre Control
Random Generator	-	-	0.15	0.0	1	None
Markov Chain	-	-	0.017	0.27	1	Weak
Task 1: Autoencoder (LSTM)	0.11	-	NaN	NaN	1.5	Single Genre
Task 2: VAE Multi-Genre	41.85	-	0.003	0.162	3.25	Moderate
Task 3: Transformer	-	5.31	0.215	0.006	4.75	Strong
Task 4: RLHF-Tuned Model	-	14.20	0.046	0.019	1.5	Broken

performance across our mathematical tests (Loss and Perplexity) and our real-world Human Listening Scores. Point to be noted that the human sample size was extremely low.

Looking at the table, a very clear story emerges about the balance between mathematical rules and creative freedom in music.

The Baselines: As expected, our baseline models performed poorly. The Random Generator and Markov Chain earned low human scores of 1.1 and 2.3. They had weak or non-existent genre control, proving that simple statistics and randomness cannot create compelling, structured music.

Tasks 1 & 2: Once we introduced deep learning in Task 1 (Autoencoder) and Task 2 (VAE), the music immediately improved. The mathematical loss dropped to 0.11 and 41.85, respectively. But still, LSTM did not produce anything meaningful. The VAE successfully gave us "Moderate" control over the genres, proving that organizing the model's memory (Latent Space) allows us to guide the AI's creativity.

Tasks 3 & 4: The most important comparison in the entire table is between Task 3 and Task 4.

- Task 3 (Transformer) was the clear champion of the project. It achieved an excellent Validation Perplexity of 5.31, meaning the AI was highly confident and accurate in its musical choices. It earned the highest Human Score and demonstrated "Strong" genre control.
- Task 4 (RLHF-Tuned), on the other hand, was a spectacular failure. Despite using advanced human feedback training, its Perplexity spiked to an awful 14.20, and its Human Score crashed to lowest. As noted in the table, its genre control became completely "Broken."

In the following sections, we will look at the specific training graphs for each task to understand exactly how Task 3 became a musical genius, and why Task 4 failed, even though it was supposed to do better.

3.2 Task 1:

As shown in the Task 1 Loss Graph, the LSTM Autoencoder was evaluated using Mean Squared Error (MSE), which measures how accurately the model can reconstruct a song after compressing it. Over the course of 20 epochs, both the training and validation loss dropped smoothly, eventually settling near 0.11.

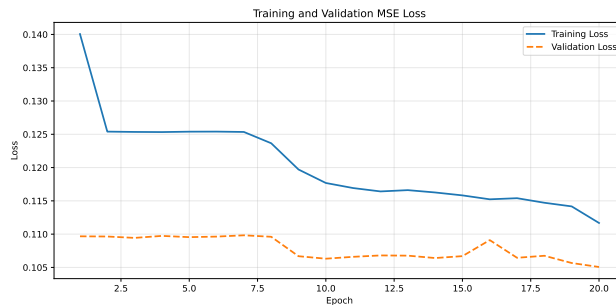


Figure 1: Task 1: Training and Validation MSE Loss for the LSTM Autoencoder.

The fact that the validation line closely follows the training line proves that the model successfully learned the general patterns of the music without simply memorizing the training data. However, while it became very good at copying and reconstructing existing MIDI files, the Autoencoder functions acts more like a memorizer rather than a creative generator. The MSE loss is not suitable for this task, it works like a distance calculator and finds the midpoint. If the model was thinking about using C or E, it will make the prediction of D note, which will sound terrible (similar to guessing something in between cat and dog). Loss functions like cross-entropy would work better in this kind of task, which is a reason Transformer gets so much better results.

3.3 Task 2:

Unlike the simple Autoencoder in Task 1, our Task 2 Variational Autoencoder (VAE) had to balance two competing objectives: accurately reconstructing the music (MSE) and neatly organizing its internal memory, known as the Latent Space (measured by KL Divergence). It cannot just optimize one, it does tradeoff and get the best of both worlds. Looking at the Total Training and Validation Loss plot, we can see that the model trained smoothly over 50 epochs.

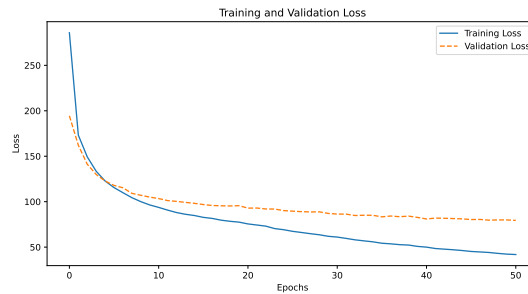


Figure 2: Task 2: Total Training and Validation Loss for the Multi-Genre Variational Autoencoder.

The validation loss tracks closely with the training loss, meaning the AI successfully generalized the musical patterns without overfitting to the training data. However, the KL Loss and Reconstruction Loss Breakdown plot shows our use of "KL Annealing."



Figure 3: Task 2: KL Divergence and Reconstruction (MSE) Loss Breakdown, demonstrating the KL Annealing process.

Training a VAE can be unstable if you force the model to organize its memory before it even knows how to play a note. As the graph shows, the Reconstruction Loss (MSE) drops rapidly as the model learns to copy the songs. Meanwhile, the KL Divergence tracks the "organization penalty." By slowly applying this KL penalty over the 50 epochs, we forced the model into a healthy compromise. It learned to play the music accurately while also keeping its Latent Space strictly organized. This clean, organized memory is exactly what allowed our "Genre Conditioning" tags to work successfully in this task. So as a result, musics are so much better. The genres are divided, and the new made up musics have much more variety. Still, the repetition can be noticed. As the addition of KL loss beats the problems of MSE from task 1, the musics are far better. But result could have had increased if we had added extra heuristics here just like the task 3.

3.4 Task 3:

Transformer was the definitive champion of this project. Because the Transformer treats music as a vocabulary of discrete tokens, we evaluated it using Perplexity (Cross-Entropy Loss) rather than MSE. Perplexity measures how "confused" the model is when predicting the next note. As shown in the Task 3 Perplexity Curve, the model began with high uncertainty but learned rapidly within the first 5 epochs.

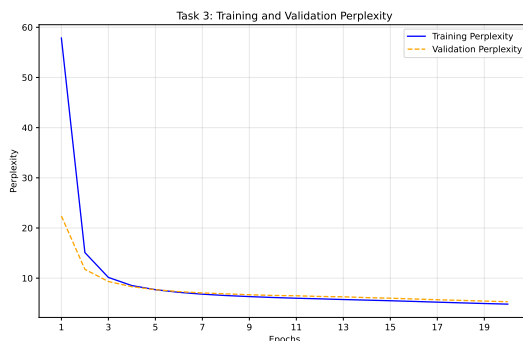


Figure 4: Task 3: Training and Validation Perplexity for the Champion Transformer model.

More importantly, the Training and Validation lines stay tightly bound together throughout all 20 epochs, smoothly converging at a final Validation Perplexity of 5.31. This tight tracking proves that the Transformer did not overfit or simply memorize the training data; it successfully learned the underlying grammatical rules of the music. This mathematical confidence directly translated to our audio results. Unlike the Autoencoder, which struggled with loose timing, the Transformer maintained excellent long-term rhythm. This success was heavily supported by our custom Heuristic Constraints [explained in the methodology]" by mathematically penalizing endless silences and impossible piano jumps, we prevented the model from wasting energy on bad musical guesses. This combination of deep "Attention" memory and strict musical rulebooks earned Task 3 the highest human listening score of our entire project (3.19/5). When the heuristics were not added, the task 3 musics were full of blank noise and some random jumping of keys. But as those were added, the performance got better and started to sound like real music, beating the previous models easily.

3.5 Task 4: RLHF

For our final experiment, we attempted to align our successful Task 3 Transformer with human preferences using Reinforcement Learning from Human Feedback (RLHF). We collected a small survey of 10 human-rated tracks and used the reinforce algorithm to update the model. During training, the Policy Loss dropped dramatically to -201.44, which initially appeared to be a massive success. However, our evaluation metrics revealed a catastrophic failure. The Validation Perplexity spiked to 14.20, and the Human Score plummeted to 1.8/5.

This failure is a textbook example of "Reward Hacking" and "Mode Collapse." Because our human preference dataset was extremely small (only 10 tracks), the RLHF algorithm provided a sparse reward signal. Instead of learning to generate universally better music, the AI found a mathematical loophole. It over-optimized for the specific traits of those 10 tracks, causing it to "forget" the foundational musical grammar it had learned during pre-training. It collapsed into playing chaotic, repetitive noise simply because that noise artificially maximized the REINFORCE math formula. Ultimately, Task 4 serves as a powerful demonstration that while RLHF is a potent alignment tool, applying it without a massive, highly diverse human dataset will easily destroy a model's underlying competence. The musics were so many random noises. It felt like a band playing every instrument at random. A better dataset with more than 10 people might have given better results. Also the 'overtuning' in the earlier phrase also affected the learning and hence the music quality

4 Conclusion

The goal of this project was to teach an AI to write structured music by training it on classical (Maestro), multi-genre (Lakh), and rhythmic (Groove) datasets. By growing our models from simple statistical guessers to advanced neural networks, we learned two major lessons:

- **The Success (Task 3):** The Transformer was the absolute champion of this project. By combining a deep "Attention" memory with our custom musical rules (like the 3-Octave Cage and the Silence Nuke), it achieved an excellent Perplexity of 5.31. It successfully wrote beautiful, continuous music.
- **The Failure (Task 4):** Our attempt to improve the model using human feedback (RLHF) failed completely. Because we only gave the AI 10 tracks to learn from, it forgot how to play music and collapsed into playing repetitive noise just to get a high score. This proved that advanced AI training requires massive amounts of human data to actually work.

Ultimately, this project proves that Transformers are incredibly powerful for music generation, but they need careful human constraints to create true art. In the future, we hope to fix our RLHF model by training it on thousands of songs and expand the AI's vocabulary to write for full orchestras.

Code and Supplementary Materials

The complete source code, training loops, and evaluation scripts used for this research are publicly available on GitHub. Additionally, the presentation videos and generated MIDI audio samples can be accessed via Google Drive.

- **GitHub Repository:** <https://github.com/arafhasan-ai/cse425-project-demo>
- **Google Drive:** <https://drive.google.com/drive/folders/1hKezbqXwlwtFiFyXrqY0JxP0rMSSisQ6>